

C Language Review

COP 3520C – Computer Science I

Spring 2026

Yancy Vance Paredes, PhD

Disclaimer

- The following slide deck is not meant to serve as a comprehensive review of C
- Use this as a checklist to indicate your familiarity with the terms being discussed
- If you do not know or understand a particular concept, try your best to learn it; brushing it off will not be beneficial

C Programming Language

- It is a **compiled** and a structured programming language
- Compiled because our high-level source codes are translated into machine codes

Anatomy of a C Program

- It has preprocessor directives
- Must have a **main()** function because it is the **entry point**
- C doesn't rely on indentation
- Therefore, a group of statements are enclosed in { }

Types of Error

- Syntax
 - Runtime
 - Logical or Semantic
- 

Structured Programming Paradigm

- Sequential Structure
- Selection Structure
- Repetition Structure
- Modularity

Sequential Structure

- We wrote simple statements
- All executable statements end with a ; (**semicolon**)
- We wrote our initial Hello World C Program
- The function we used was the **printf()** function

Escape Sequences

- Special sequences of characters that have a special meaning to C
- For example, the newline character `\n`

Key Topics

- Data and Memory
- Pointers
- Arrays
- Strings
- Structures

Data and Memory

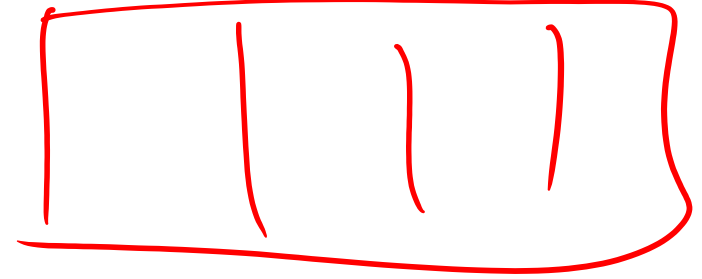
data type identifier ;

- C supports various **data types**

char, int, float, double

- A data type uses a particular **memory size** (machine-dependent)
- To know **how much memory** (in bytes) a data type (variable or value) requires, use the **sizeof** operator

Variable



- An **alias** to a **memory location** where data is stored
- Needs to be **declared**
- Must specify the **data type**

Pointer

- A **variable** that stores **memory address** as its value

- Syntax:

`int * ptr;`

`int * ptr;`

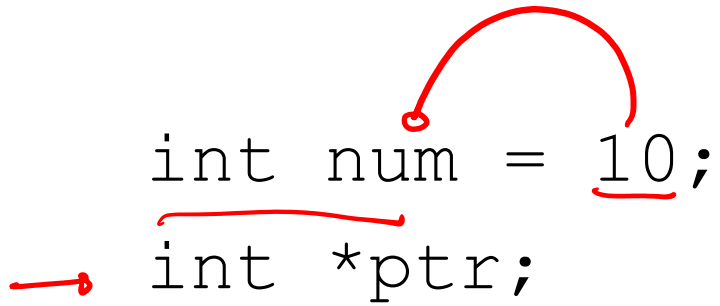
`int * ptr;`

0x FF

Practice /1

Analyze the following code fragment:

```
int num = 10;  
int *ptr;
```



num

ptr

Memory Location	Memory Content
#A	10
#B	?
#C	
#D	
#E	

Practice /2

num

Analyze the following code fragment:

```
int num = 10;
```

```
int *ptr;
```

```
ptr = #A &num;
```

num →
ptr →

Memory Location	Memory Content
#A	10
#B	? #A
#C	
#D	
#E	

Practice /3

*int * ptr, ptr2;*

Analyze the following code fragment:

✓ `int num = 10;`

✓ `int *ptr, *ptr2;`

✓ `ptr = #`

ptr2 = ptr;

num →
ptr →
ptr2 →

Memory Location	Memory Content
#A	10
#B	#A
#C	/ #A
#D	.
#E	.

Notes

- When you want to use the memory value in your code, you simply use the name of the pointer (without the `*`)
- Including the `*` to the variable name will change its meaning

Printing Values

- We can use `printf()` to print the value of a pointer (recall: this value is a memory location or an address)
- Use `%p` as the format specifier

Pointer Operators /1

& known as **address-of operator**

What is the address of this variable?

Example: `&num`

Pointer Operators /2

Russ

* known as **dereference operator**

Access [or manipulate] the value stored at this memory location

Example: `*ptr` (Different from the * you see in the declaration)

The diagram illustrates the concept of dereferencing. It shows a variable `ptr` (underlined) containing a memory address (underlined). This address is then dereferenced using the `*` operator (underlined) to access the value stored at that memory location (underlined). The diagram is drawn with red lines and includes a red asterisk and a semicolon at the end of the first line.

```
int *ptr, num; ✓
int *ptr2, num2; ✓
```

```
num = 10; ✓
ptr = &num; ✓
num2 = 50; ✓
ptr2 = &num2; ✓
```

```
printf("%d %d\n", num, num2); ✓
*ptr = 20;
printf("%d %d\n", num, num2);
ptr2 = ptr;
printf("%d %d\n", num, num2);
*ptr2 = 90;
printf("%d %d\n", num, num2); ✓
ptr = &num2;
printf("%d %d\n", num, num2);
*ptr = 70;
printf("%d %d\n", num, num2);
```

In the end, which is **greater**? num or num2?

Memory Location	Memory Content
#A	7 #B #D
#B	1 10 20 90
#C	1 #D #B
#D	7 50 70
#E	

✓ ptr →
 ✓ num →
 ptr2 →
 num2 →

Segmentation Fault /1

- **Run-time error** due to memory access violation
- For example, dereferencing an invalid memory location
- A **best practice** is to check first if the pointer **is non-NULL**
- It is **your responsibility** to ensure this is the case!

Segmentation Fault /2

NULL X '\0'

- Initialize a pointer to **NULL** to point to “nothing” (actually it is 0)

```
int *ptr = NULL;
```

- Check pointer before dereferencing it (i.e., **defensive**)

```
if (ptr != NULL) {  
    // safe to proceed  
}
```

Pointer to a Pointer

`printf("%d %p %p\n", x, y, z);`
 %d **z

```
int x, *y, **z;
```

```
x = 10;
y = &x;
z = &y;
```

x →
y →
z →

Memory Location	Memory Content
#A	10
#B	#A
#C	#B
#D	
#E	

```
// print contents of x, y, z

1 // is x == *y ? yes
2 // is x10 == *z#A? no
3 // is #A == *z#A? yes
   // is #B == z#B? yes
   // is &x#A == **z10? no
```

* (* (#B))
 * (#A) → 10

Why Pointers?

- C wants to provide you the ability to **directly manipulate memory**
- Efficiently share data **between functions**
- Refer to large data structures in a compact way
- We will be performing dynamically memory allocation

Function Parameters

- **Pass by value**

- Function receives a **copy of the values of the variables** being passed
- *In a way, not the true sense of “shared”*

- **Pass by reference**

- Function receives **the address of the variables** being passed
- *In some sense, the variable is being “shared”*

Analyze `square_v1` function

```
void square_v1(int x)
{
    x = x * x;
}
```

```
int main(void)
{
    int num = 10;

    printf("%d\n", num);
    square_v1(num);
    printf("%d\n", num);

    return 0;
}
```

Analyze `square_v2` function

```
void square_v2(int *x)
{
    *x = (*x) * (*x);
}
```

```
int main(void)
{
    int num = 10;

    printf("%d\n", num);
    square_v2(&num);
    printf("%d\n", num);

    return 0;
}
```

Comparison of the Two Functions

```
void square_v1(int x)
{
    x = x * x;
}
```

```
// function call
square_v1(num);
```

```
void square_v2(int *x)
{
    *x = (*x) * (*x);
}
```

```
// function call
square_v2(&num);
```

Notes

- `square_v1` demonstrated **pass by value**
- `square_v2` demonstrated **pass by reference**
- Recall the concept of **variable scope**?

Variable Scopes

```
void square_v1 (int num)
{
    num = num * num;
}
```

```
// function call
square_v1 (num) ;
```

```
void square_v2 (int *num)
{
    *num = (*num) * (*num) ;
}
```

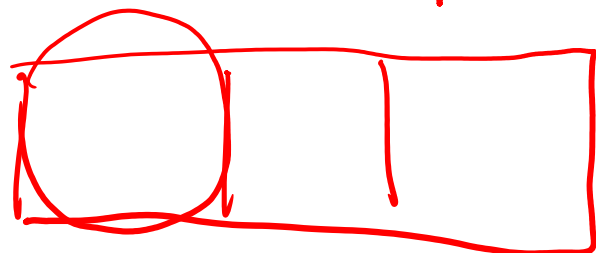
```
// function call
square_v2 (&num) ;
```

Card c;

Card *p = &c;

Card **q = &p;

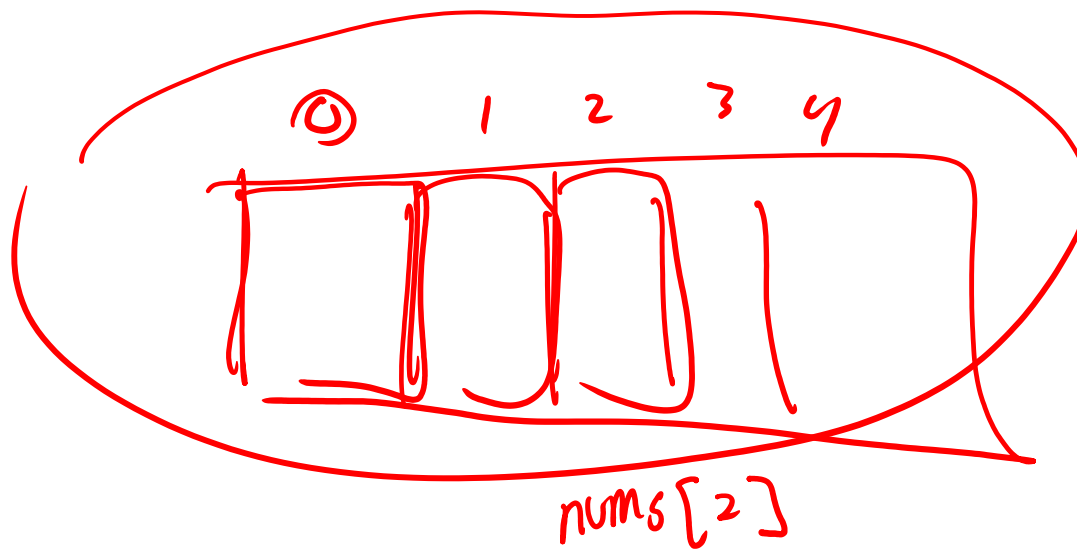
(+q) → value;



.value

&hand[i].value

hand[i].suit



int nums[5];

Card hand[3];

sizeof(int)

4 * 5 = 20

Array /1

- Think of it as a collection of “variables” (i.e., **elements**)
- All the elements have the **same data type**
- Elements are **contiguously stored** in the memory
- Therefore, we can **directly access** elements

Array /2

- How to declare an array?

```
int data[5];
```

↓
data[0]
data[1]
data[2]
data[3]
data[4]

Memory Location	Memory Content
#A	
#B	
#C	
#D	
#E	

- How many elements can this array hold?
- How do you access an element in the array? Say, 3rd element?*

```
data[2]
```

Array /3

- Another way to **initialize an array**

↓
data[0]
data[1]
data[2]
data[3]
data[4]

Memory Location	Memory Content
#A	87
#B	99
#C	75
#D	88
#E	93

```
int data[] = {87, 99, 75, 88, 93};
```

Iterating over an Array

```
int data[5] = {87, 99, 75, 88, 93};  
  
for(int i = 0; i < 5; i++) {  
    printf("%d ", data[i]);  
}
```

Practice

An **array's name** *decays* into the **address of its first element** *

```
int data[] = {87, 99, 75, 88, 93};
```

#A

```
// is data == &data[0] ? yes
```

#A *#A*

Since **data** is an address, we can use the dereference operator

```
// print *data
```

87

data *printf("%d", *data)*

Pointer Arithmetic /1

We can further extend the following notation:

```
// *(data+1) == data[1]
```

How about the 3rd element?

```
// *(data+2) == data[2]
```

Pointer Arithmetic /2

How about the 1st element?

```
// *(data+0) == data[0] == *(data) == *data
```

How about the nth element?

```
// *(data+(n-1)) == data[(n-1)]
```

Caution about precedence!

```
// *(data+(n-1)) != data[(n-1)] // Why?
```

87 +

Iterating over an Array (Pointer)

```
int data[5] = {87, 99, 75, 88, 93};
```

```
for(int i = 0; i < 5; i++) {  
    printf("%d ", *(data+i));  
}
```

$\text{data}[i]$ vs $*(\text{data}+i)$
 $\&\text{data}[i] == (\text{data}+i)$

Discussion

For a simple 1D pointer parameter:

```
arr[i] == *(arr + i)
```

int *
arr
+ i
+ 1

Refer to the visualization on the next slide

dereference
the sample
addresses

int arr[5]

OK
sample
addresses
in hex decimal

<code>*(arr + 0)</code>	<code>*(arr + 1)</code>	<code>*(arr + 2)</code>	<code>*(arr + 3)</code>	<code>*(arr + 4)</code>
<code>*(0xAA + 0)</code>	<code>*(0xAA + 1)</code>	<code>*(0xAA + 2)</code>	<code>*(0xAA + 3)</code>	<code>*(0xAA + 0)</code>
<code>*(0xAA)</code>	<code>*(0xAE)</code>	<code>*(0xB2)</code>	<code>*(0xB6)</code>	<code>*(0xBA)</code>
<code>arr[0]</code>	<code>arr[1]</code>	<code>arr[2]</code>	<code>arr[3]</code>	<code>arr[4]</code>
10	20	30	40	50
<code>&arr[0]</code>	<code>&arr[1]</code>	<code>&arr[2]</code>	<code>&arr[3]</code>	<code>&arr[4]</code>
<code>0xAA</code>	<code>0xAE</code>	<code>0xB2</code>	<code>0xB6</code>	<code>0xBA</code>
<code>0xAA + 0</code>	<code>0xAA + 1</code>	<code>0xAA + 2</code>	<code>0xAA + 3</code>	<code>0xAA + 4</code>
<code>arr + 0</code>	<code>arr + 1</code>	<code>arr + 2</code>	<code>arr + 3</code>	<code>arr + 4</code>

access the
value of
an element

addresses
of each
elements

Discussion

- When you **pass an array to a function**, is it effectively a pass by value OR pass by reference?
- What is the implication of this?

Your Turn!

Write a function named **reset()** that takes a pointer to an array of integers and the size of the array as parameters and sets all elements to 0.

```
void reset(int *arr, int size) {
```

Challenge

Write a function named **flip()** that takes a pointer to an array of characters and the size of the array as parameters and converts all uppercase letters to lowercase and vice versa. Note that the array may contain non-letters.

```
void flip(char *arr, int size) {  
  
  
  
  
  
  
}
```

String /1

- An array of characters
- Terminated by the ' \0 ' (null character; **null-terminated**)
- The null character **is not** part of the string

String /2

Visualize the following statement:

```
char word[10] = "dog";
```

0	1	2	3	4	5	6	7	8	9
'd'	'o'	'g'	'\0'						

What is the length of `word`? How to print each `char` using a loop?

String /3

- A string's length can be obtained using the **strlen()** function
- Take note, the null character is **not counted**
- More functions are defined in `string.h` library

Function	Purpose
<code>strcpy()</code>	Makes a copy of source, a string, in the character array accessed by dest:
<code>strncpy()</code>	Makes a copy of up to n characters from source in dest: <code>strncpy(dest, source, 5)</code> stores the first five characters of the source and does NOT add a null character.
<code>strcat()</code>	Appends source to the end of dest: <code>strcat(dest, source)</code>
<code>strncat()</code>	Appends up to n characters of source to end of dest, adding the null character if necessary.
<code>strcmp()</code>	Compares s1 and s2 alphabetically. Returns a negative value if s1 should precede s2, a zero if strings are equal, and a positive value if s2 should precede s1 in an alphabetized list. <code>strcmp(s1,s2)</code>
<code>strncmp()</code>	Compares the first n characters in s1 and s2 returning positive, zero, and negative values like <code>strcmp</code> .
<code>strlen()</code>	Returns the number of characters in s, not counting the terminating null. <code>strlen(s)</code>
<code>strtok()</code>	Breaks the parameter string into tokens finding groups of characters separated by any of the delimiter characters. Each group is separated with '\0'.
<code>strchr()</code>	Returns a pointer to the first location of a character located in the string. Null is returned if character is not found.
<code>strpbrk()</code>	Return a pointer to the first location in the strings that holds any character found in another string.
<code>strrchr()</code>	Returns a pointer to the last occurrence of a character in the string. Null is returned if character not found.
<code>strstr()</code>	Returns a pointer to the first occurrence of string s2 in string s1. Null is returned if character not found.

Array of Strings

- Recall that a string is an array of characters
- If you have an array of string, essentially you have a 2D array

Practice

Write a program that prompts the user to enter a string of lowercase letters, then prints the position of each letter in the alphabet (e.g., 'a' = 1, 'b' = 2, ..., 'z' = 26).

Sample Run

Enter Word: hello

8 5 12 12 15

Sample Run (Simplified)

Enter Word: abc

1 2 3

Structure

- Allows you to define your **own data type** where related variables (usually, different data types) are logically grouped
- Use the **struct** keyword
- Oftentimes, used alongside with **typedef** to create an alias
- Access **members** (fields or components) using **.** (**dot operator**)

Practice /1

Define a **Person** struct that has two members: **name** and **age**.
Declare a **Person** variable and set the values to “John” and 20, respectively.

Practice /2

Declare an array of **Person** where the size is 10.

Practice /3

Define a function that takes two parameters: an array **Person** and an integer representing the number of elements in the array. The function should print the name and age of each person, with one person's details per line.

```
void introduce(Person *arr, int size) {
```


Structures and Pointers

- When you have a pointer to a structure, you **dereference** it first
- However, a **shortcut** is to use the **-> arrow operator**

$(*ptr).field$ is same as $ptr \rightarrow field$

A Member that is a Pointer

- What we know about pointers will still be true in the following:

```
typedef struct Person_s {  
    char *name;  
    int age;  
} Person;
```

- This convention will be common in our course, along with **dynamic memory allocation**.

A Member that is another Structure

```
typedef struct Address_s {  
    char *city;  
    int zip_code;  
} Address;
```

```
typedef struct Person_s {  
    char *name;  
    int age;  
    Address mail_address;  
    Address billing_address;  
} Person;
```

Person p;

p.mail_address.zip_code = 10;

Notes

- It is important that understand when you can use the arrow operator when dealing with structures
- Our codes later in the course will be heavily rely on this concept

Questions?